COMPUTER ORGANISATION ARCHITECTURE

[UCS4012]

TOPIC :- ARITHMETIC LOGIC CALCULATOR

INNOVATIVE I.T

REPORT FILE

SUBMITTED BY:-

NAME:- ABHISHEK KUMAR GUPTA

ROLL NO:- 202410101150226

COURSE:- B.TECH CSE (DS+AI)

GROUP:-47,48

SUBMITTED TO:-

DR. AKHILESHH KR. GUPTA

Arithmetic Logic Unit (ALU) Simulation

1. Introduction

The **Arithmetic Logic Unit (ALU)** is a fundamental building block of the Central Processing Unit (CPU). It is the digital circuit responsible for performing integer arithmetic and logical operations. In modern computing, the ALU receives data from processor registers, processes it based on an instruction opcode, and stores the result back in an output register.

2. Objectives

- To simulate the functional behavior of an ALU using high-level programming.
- To understand the role of **Control Signals (OpCodes)** in selecting operations.
- To implement basic arithmetic (Addition, Subtraction) and bitwise logic (AND, OR, XOR, NOT).

3. ALU Architecture and Design

An ALU consists of two main subsections:

1. **Arithmetic Unit:** Performs operations like addition ($A + B$) and subtraction ($A - B$). In physical hardware, this is typically constructed using a series of **Full Adders**.
2. **Logic Unit:** Performs bit-by-bit logical operations. These are independent of carries or borrows between bits.
3. **Multiplexer (MUX):** Acts as the selection logic. Based on the n-bit OpCode, the MUX selects which unit's output will be sent to the final result bus.

4. Implementation Logic (Scilab)

The simulation utilizes a modular function approach. The inputs are two operands and a selection variable.

Operation Mapping:

OpCode	Operation	Description
000 (0)	**ADD**	Returns the sum of A and B
001 (1)	**SUB**	Returns the difference (A - B)
010 (2)	**AND**	Bitwise logical AND
011 (3)	**OR**	Bitwise logical OR
100 (4)	**XOR**	Bitwise logical Exclusive OR
101 (5)	**NOT**	Bitwise inversion of Input A

5. Scilab Code Implementation

The following code simulates the logic gates and arithmetic circuits within a virtual ALU environment.

Scilab:

```
// Arithmetic Logic Unit (ALU) Simulation
// Computer Organization & Architecture Project

function [result]=alu_operation(A, B, opcode)
    // opcode: 0=Add, 1=Sub, 2=AND, 3=OR, 4=XOR, 5=NOT (on A)

    select opcode
    case 0 then
        result = A + B;
        disp("Operation: Addition");
    case 1 then
        result = A - B;
        disp("Operation: Subtraction");
    case 2 then
        result = bitand(A, B);
        disp("Operation: Bitwise AND");
    case 3 then
        result = bitor(A, B);
        disp("Operation: Bitwise OR");
    case 4 then
        result = bitxor(A, B);
        disp("Operation: Bitwise XOR");
    case 5 then
        result = bitnot(A);
        disp("Operation: Bitwise NOT (Input A)");
    else
        disp("Invalid OpCode!");
        result = %nan;
    end
endfunction

// --- Main Program Execution ---
clc;
printf("--- Scilab ALU Project ---\n");

// Input Values (Assume 8-bit integers for simulation)
valA = input("Enter Integer A: ");
valB = input("Enter Integer B: ");

printf("\nSelect OpCode:\n0: ADD | 1: SUB | 2: AND | 3: OR | 4: XOR | 5: NOT\n");
op = input("Enter OpCode (0-5): ");

// Execute ALU
res = alu_operation(valA, valB, op);

// Output result
printf("\n---------------------------\n");
printf("Result: %d\n", res);
printf("--------------------------\n");
```

6. Observations and Results

- **Arithmetic Accuracy:** The Scilab + and - operators effectively simulate the behavior of binary adders and subtractors.
- **Logic Bitwise Handling:** Using Scilab's internal bitand and bitor functions allows the simulation of gate-level logic where each bit position is calculated independently.
- **Selection Logic:** The select structure in the code successfully mimics the hardware behavior of a **4-to-1 or 8-to-1 Multiplexer**, ensuring only the desired operation reaches the output.

7. Conclusion

This project demonstrates that an ALU's complexity lies in its selection logic. While the individual operations (like AND or ADD) are simple, the coordination via OpCodes allows the CPU to perform multifaceted tasks. Scilab provides an efficient environment for testing these architectural concepts before moving to hardware description languages like Verilog or VHDL.

Would you like to expand this report with a section on how "Flag Registers" (like Carry, Zero, and Negative flags) are implemented?